Module 3: Data Organization and Manipulation using Collections

Premanand S

Assistant Professor
School of Electronics Engineering
Vellore Institute of Technology
Chennai campus

premanand.s@vit.ac.in

September 2, 2025

In module 3,

- Data Handling Strategies Using Lists, Tuples, Dictionary and Set
- Data Comprehension
- Data Modification
- Grouping and Categorization
- Iterators
- Data Selection - Ordered, Unordered and Unique Data
- Searching Techniques: Linear Search, Binary Search
- Sorting Techniques: Bubble Sort, Selection Sort- Insertion Sort- Quick Sort- Merge Sort.

Collection Data Types in Python

- Special data structures that allow us to store, organize, and manipulate groups of data (multiple values) in a single variable.
- They are the backbone of data handling strategies.
- Types,
  - List
  - Tuple
  - Dictionary
  - Set

What is Data Handling Strategy?

- Data Handling Strategies are the methods and techniques used to store, organize, access, modify, and process data effectively using suitable data structures and algorithms.
- In Python, strategies involve using:
  - Data structures $\rightarrow$ Lists, Tuples, Dictionaries, Sets
  - Techniques $\rightarrow$ Iteration, Comprehension, Searching, Sorting, Grouping
- Analogy: It's like deciding where to keep your clothes (cupboard, drawer, hanger) and how to find them later (by color, type, or season).

Why Do We Need Data Handling Strategies?

- Efficiency – To save time while accessing or modifying data.
- Organization – To keep data in a structured way.
- Memory Management – Choosing the right structure avoids unnecessary memory use.
- Problem-Solving – Different problems need different handling approaches.

Importance of Data Handling Strategies

- Improves performance of programs (fast searching, optimized sorting).
- Reduces complexity in coding by using the right structure.
- Prevents errors like duplicates or unordered data issues.
- Scalability – Handles large datasets more efficiently.
- Reusability & Readability – Easier for others to understand and maintain.

Characteristics - LIST

LIST

- Analogy - Shopping
- Characteristics
  - Mutable
  - Linear data structures
  - Mixed data type
  - Variable length
  - Zero-based indexing (Negative indexing, One-based, associative indexing, custom indexing)

List Data Type

Example (Ordered)

```
fruits = ["apple", "banana", "cherry"]
print(fruits[0])
print(fruits[2])
```

List Data Type

Example (Mutable (Changeable))

```
fruits = ["apple", "banana", "cherry"]
fruits[1] = "grape"
fruits.append("orange")
fruits.remove("apple")
print(fruits)
```

List Data Type

Example (Allows Duplicates)

```
numbers = [1, 2, 2, 3, 4, 4, 4]
print(numbers)
```

List Data Type

Example (Heterogeneous (Mixed Data Types))

```
data = [10, "Hello", 3.14, True]
print(data)
```

List Data Type

Example (Nested Lists)

```
matrix = [[1, 2, 3], [4, 5, 6], [7, 8, 9]]
print(matrix[1][2])
```

List Data Type

Example (Supports Negative Indexing)

```
fruits = ["apple", "banana", "cherry"]
print(fruits[-1])
print(fruits[-2])
```

List Data Type

Example (Supports Slicing)

```
fruits = ["apple", "banana", "cherry", "orange", "mango"]
print(fruits[1:4])
print(fruits[:3])
print(fruits[-3:])
```

List Data Type

Example (Dynamic (Resizable))

```
nums = [1, 2]
nums.append(3)
nums.extend([4, 5])
nums.pop()
print(nums)
```

How to create - LIST

List Data Type

Example (Using Square Brackets [ )

]

```
fruits = ["apple", "banana", "cherry"]
print(fruits)
```

List Data Type

Example (Using list() Constructor)

```
a = list()
print(a)  # []

b = list((1, 2, 3))
print(b)

c = list("Python")
print(c)

d = list(range(5))
print(d)
```

List Data Type

Example (List Comprehension)

```
squares = [x*x for x in range(5)]
print(squares)

evens = [x for x in range(10) if x % 2 == 0]
print(evens)
```

List Data Type

Example (Using * Operator (Repetition))

```
zeros = [0] * 5
print(zeros)

nested = [[1, 2]] * 3
print(nested)
```

List Data Type

Example (split() (from Strings))

```
text = "Python is fun"
words = text.split()
print(words)
```

List Data Type

Example (map() Function)

```
nums = list(map(int, "12345"))
print(nums)
```

List Data Type

Example (From Another Collection)

```
s = {10, 20, 30}
lst1 = list(s)
print(lst1)

d = {"a": 1, "b": 2}
lst2 = list(d)
print(lst2)
```

How to access - LIST

List Data Type

Example (Access by Index (Positive Indexing))

```
fruits = ["apple", "banana", "cherry", "orange", "mango"]

print(fruits[0])
print(fruits[2])
print(fruits[4])
```

List Data Type

Example (Access by Index (Negative Indexing))

```
fruits = ["apple", "banana", "cherry", "orange", "mango"]

print(fruits[-1])
print(fruits[-2])
print(fruits[-5])
```

List Data Type

Example (Access by Slicing)

```
fruits = ["apple", "banana", "cherry", "orange", "mango"]

print(fruits[1:4])
print(fruits[:3])
print(fruits[2:])
print(fruits[::2])
print(fruits[::-1])
```

List Data Type

Example (Access by Loops)

```
fruits = ["apple", "banana", "cherry", "orange", "mango"]

for fruit in fruits:
    print(fruit)

for i in range(len(fruits)):
    print(i, fruits[i])
```

List Data Type

Example (Access by List Comprehension)

```
fruits = ["apple", "banana", "cherry", "orange", "mango"]

first_letters = [fruit[0] for fruit in fruits]
print(first_letters)
```

List Data Type

Example (Access by enumerate())

```
fruits = ["apple", "banana", "cherry", "orange", "mango"]

for index, fruit in enumerate(fruits):
    print(index, fruit)
```

List Data Type

Example (Access by random module)

```
fruits = ["apple", "banana", "cherry", "orange", "mango"]

import random
print(random.choice(fruits))
```

INDEXING - LIST

List Data Type

Example (Positive Indexing)

```
fruits = ["apple", "banana", "cherry", "orange", "mango"]

print(fruits[0])
print(fruits[2])
print(fruits[4])
```

List Data Type

Example (Negative Indexing)

```
fruits = ["apple", "banana", "cherry", "orange", "mango"]

print(fruits[-1])
print(fruits[-2])
print(fruits[-5])
```

List Data Type

Example (Indexing range)

```
fruits = ["apple", "banana", "cherry", "orange", "mango"]

print(fruits[10])
```

List Data Type

Example (for nested list)

```
data = [10, 20, [30, 40, 50], 60]

print(data[0])
print(data[2])
```

List Data Type

Example (for nested list)

```
data = [10, 20, [30, 40, 50], 60]

print(data[2][0])
print(data[2][1])
print(data[2][2])
```

List Data Type

Example (for nested list)

```
data = [10, 20, [30, 40, 50], 60]

print(data[-2][-1])
print(data[-2][-3])
```

List Data Type

Example (for nested list)

```
matrix = [
    [1, 2, 3],
    [4, 5, [6, 7]],
    [8, 9]
]

print(matrix[1][2][0])
print(matrix[1][2][1])
```

List Data Type

Example (Indexing with Loop)

```
fruits = ["apple", "banana", "cherry", "date"]

# Loop with indexing
for i in range(len(fruits)):
    print(f"Index {i} → {fruits[i]}")
```

List Data Type

Example (Indexing with Condition)

```
numbers = [10, 25, 30, 45, 50]

# Print only elements greater than 30 with their index
for i in range(len(numbers)):
    if numbers[i] > 30:
        print(f"Index {i} → {numbers[i]}")
```

List Data Type

Example (Using enumerate() Function)

```
colors = ["red", "green", "blue"]

for index, value in enumerate(colors):
    print(f"Index {index} → {value}")
```

List Data Type

Example (Using enumerate() Function with condition)

```
marks = [55, 80, 92, 40, 76]

# Find students who scored >= 75
for index, score in enumerate(marks):
    if score >= 75:
        print(f"Student {index} scored {score}")
```

SLICING - LIST

List Data Type

Example (Slicing)

```
list[start : stop : step]

start = starting index (inclusive). Default = 0.

stop = ending index (exclusive). Default = len(list).

step = difference between indices. Default = 1.
```

List Data Type

Example (Slicing)

```
numbers = [10, 20, 30, 40, 50, 60]

print(numbers[1:4])
print(numbers[:3])
print(numbers[2:])
```

List Data Type

Example (Negative Slicing)

```
nums = [1, 2, 3, 4, 5, 6]

print(nums[-4:-1])
print(nums[:-2])
print(nums[-3:])
```

List Data Type

Example (Step in Slicing)

```
letters = ['a', 'b', 'c', 'd', 'e', 'f']

print(letters[::2])
print(letters[1::2])
print(letters[::3])
```

List Data Type

Example (Negative Step)

```
data = [100, 200, 300, 400, 500]

print(data[::-1])
print(data[4:1:-1])
print(data[3::-1])
```

List Data Type

Example (Omitting Start/End)

```
nums = [10, 20, 30, 40, 50]

print(nums[:])
print(nums[::])
print(nums[::-1])
```

List Data Type

Example (Complete Slice)

```
a = [7, 8, 9]
b = a[:]
print(b)
print(a is b)
```

List Data Type

Example (Slicing with Out-of-Range Index)

```
nums = [10, 20, 30]

print(nums[0:10])
print(nums[-10:2])
```

List Data Type

Example (Copy and Modify Without Changing Original)

```
nums = [1, 2, 3, 4, 5]
copy_nums = nums[:]

copy_nums[0] = 100
print(nums)
print(copy_nums)
```

List Data Type

Example (Nested List Slicing)

```
nested = [[1, 2, 3], [4, 5, 6], [7, 8, 9]]

print(nested[0][1:])
print(nested[1][:2])
print(nested[:2])
print(nested[1:][0])
print(nested[0][1:])
```

METHODS - LIST

List Data Type

Example (How to Check All Available Methods for Lists?)

```
print(dir(list))

help(list.append)
```

List Data Type

- The append() method adds a single element to the end of a list.
- Only one element at a time can be appended. If you append another list, it will be treated as a single nested element, not merged.

Example (list.append())

```
data = [1, 2, 3]

data.append(4)
data.append("hello")
data.append([7, 8])
data.append((9, 10))
data.append({11, 12})
data.append({"a": 1})

print(data)
```

List Data Type

Example (list.append())

```
squares = []
for i in range(1, 6):
    squares.append(i * i)

print(squares)
```

List Data Type

Example (list.append())

```
nums = []
n = int(input("How many numbers? "))

for i in range(n):
    val = int(input(f"Enter number {i+1}: "))
    nums.append(val)

print("Final List:", nums)
```

List Data Type

- The extend() method adds multiple elements (from another iterable like list, tuple, set, string, dict, etc.) to the end of a list.
- Unlike append(), which adds the entire object as one element, extend() unpacks the iterable and adds each item separately.

Example (list.extend())

```
nums = [1, 2]
nums.extend((3, 4, 5))
print(nums)

nums.extend({3, 4})
print(nums)

chars = ["a", "b"]
chars.extend("cd")
print(chars)
```

List Data Type

Example (list.extend())

```
data = [10, 20]
data.extend({"x": 1, "y": 2})
print(data)

list1 = []
for i in range(3):
    list1.extend([i, i+10])
print(list1)
```

List Data Type

Example (list.append() vs list.extend())

```
x = [1, 2, 3]
y = [4, 5]

x.append(y)
print("append:", x)

x = [1, 2, 3]
x.extend(y)
print("extend:", x)
```

List Data Type

- The insert() method is used to add an element at a specific position (index) in a list.
- Unlike append() (always adds at the end) and extend() (adds multiple elements), insert() gives you control over where to place the element.

Example (list.insert())

```
nums = [10, 20, 30]
nums.insert(0, 5)
print(nums)

nums = [1, 2, 3]
nums.insert(len(nums), 4)
print(nums)
```

List Data Type

Example (list.insert())

```
nums = [1, 2, 3]
nums.insert(100, 99)
print(nums)

items = [1, 2, 3]
items.insert(1, [100, 200])
print(items)

data = [10, 20]
data.insert(1, (30, 40))
print(data)
```

List Data Type

Example (list.insert())

```
chars = ["a", "b", "c"]
chars.insert(2, "z")
print(chars)

nums = [10, 20, 30, 40]
nums.insert(-1, 25)
print(nums)
```

List Data Type

- list.remove(x) removes the first occurrence of the element x from the list.
- If the element is not found → it raises a ValueError.

Example (list.remove())

```
fruits = ["apple", "banana", "cherry", "banana", "orange"]
fruits.remove("banana")
print(fruits)

numbers = [1, 2, 3, 4, 5]
numbers.remove(10)

nums = [1, 2, 2, 3, 2, 4]
while 2 in nums:
    nums.remove(2)
print(nums)
```

List Data Type

Example (list.remove())

```
data = [1, 2, [3, 4], 5]
data.remove([3, 4])
print(data)

colors = ["red", "blue", "green"]
if "yellow" in colors:
    colors.remove("yellow")
else:
    print("Element not found")
```

List Data Type

- Removes an element at a given index (default: last element).
- Returns the removed element.
- If index is out of range $\rightarrow$ raises IndexError.

Example (list.pop())

```
fruits = ["apple", "banana", "cherry"]
removed = fruits.pop()
print("Removed:", removed)
print("List now:", fruits)

numbers = [10, 20, 30, 40, 50]
removed = numbers.pop(2)
print("Removed:", removed)
print("List now:", numbers)
```

List Data Type

Example (list.pop())

```
items = [1, 2, 3]
items.pop(10)

stack = [1, 2, 3, 4]
while stack:
    print("Popped:", stack.pop())

data = [[1, 2], [3, 4], [5, 6]]
removed = data.pop(1)   # removes [3, 4]
print("Removed:", removed)
print("Remaining:", data)
```

List Data Type

- Removes all elements from the list.
- After calling → list becomes empty ([]).
- It does not delete the list object itself, only empties its content.

Example (list.clear())

```
fruits = ["apple", "banana", "cherry"]
fruits.clear()
print(fruits)

numbers = [1, 2, 3]
print("Before clear:", id(numbers))

numbers.clear()
print("After clear:", id(numbers), numbers)
```

List Data Type

Example (list.clear())

```
nested = [[1, 2], [3, 4], [5, 6]]
nested[0].clear()   # clears only first inner list
print(nested)
```

List Data Type

- Returns the index (position) of the first occurrence of a given element in the list.
- If the element is not found, it raises a ValueError.

Example (list.index())

```
fruits = ["apple", "banana", "cherry", "apple"]
print(fruits.index("apple"))

colors = ["red", "green", "blue"]
print(colors.index("yellow"))
```

List Data Type

Example (list.index())

```
nums = [10, 20, 30, 20, 40, 20]

print(nums.index(20))
print(nums.index(20, 2))
print(nums.index(20, 4, 6))

nested = [[1, 2], [3, 4], [5, 6]]
print(nested.index([3, 4]))
```

List Data Type

- Returns the number of times a given element appears in the list.
- Unlike index(), it does not raise an error if the element is not present. Instead, it returns 0.

Example (list.count())

```
fruits = ["apple", "banana", "apple", "cherry", "apple"]
print(fruits.count("apple"))

colors = ["red", "green", "blue"]
print(colors.count("yellow"))

words = ["Python", "python", "PYTHON", "python"]
print(words.count("python"))
```

List Data Type

Example (list.count())

```
nested = [[1, 2], [3, 4], [1, 2], [5, 6]]
print(nested.count([1, 2]))

chars = ['a', 'b', 'a', 'c', 'b', 'a']
unique = set(chars)

for ch in unique:
    print(f"{ch}: {chars.count(ch)}")
```

List Data Type

- Sorts the elements of the list in place (modifies the original list).
- By default $\rightarrow$ Ascending order.

Example (list.sort())

```
nums = [4, 2, 8, 1, 7]
nums.sort()
print(nums)

nums = [4, 2, 8, 1, 7]
nums.sort(reverse=True)
print(nums)
```

List Data Type

Example (list.sort())

```
words = ["banana", "apple", "cherry", "date"]
words.sort()
print(words)

words = ["Banana", "apple", "Cherry", "date"]
words.sort()
print(words)

words = ["Banana", "apple", "Cherry", "date"]
words.sort(key=str.lower)
print(words)
```

List Data Type

Example (list.sort())

```
words = ["banana", "apple", "cherry", "date"]
words.sort(key=len)
print(words)

pairs = [(2, 'b'), (1, 'a'), (3, 'c')]
pairs.sort()
print(pairs)

nums = [-4, -1, 3, -2, 5]
nums.sort(key=abs)
print(nums)
```

List Data Type

- Reverses the order of elements in the list.
- Unlike sort(), it does not sort — it only flips the sequence.

Example (list.reverse())

```
nums = [1, 2, 3, 4, 5]
nums.reverse()
print(nums)

words = ["apple", "banana", "cherry"]
words.reverse()
print(words)

mixed = [10, "hello", 3.5, True]
mixed.reverse()
print(mixed)
```

List Data Type

- Returns a shallow copy of the list.
- The new list will have the same elements, but it will be stored at a different memory location.
- Modifications in the new list will not affect the original list (for non-nested data).

Example (list.copy())

```
fruits = ["apple", "banana", "cherry"]
fruits_copy = fruits.copy()

print("Original:", fruits)
print("Copied:", fruits_copy)
```

List Data Type

Example (list.copy())

```
fruits = ["apple", "banana", "cherry"]
fruits_copy = fruits.copy()

print(id(fruits))
print(id(fruits_copy))

nums = [1, 2, 3]
nums_copy = nums.copy()

nums_copy.append(4)

print("Original:", nums)
print("Modified copy:", nums_copy)
```

List Data Type

Example (list.copy())

```
import copy

nested = [[1, 2], [3, 4]]
deep_copy = copy.deepcopy(nested)

deep_copy[0][0] = 99

print("Original:", nested)
print("Deep copy:", deep_copy)
```